TRAFFIQ 2025

April 9, 2026

AUTONOMOUS NAVIGATION PROJECT

TRAFFIQ AI Model

Online

Active

"Autonomous navigation using real-time vision and decision intelligence"

Real-time Vision

Team

NEURO DRIVE

Members

1. Archideb Chatterjee
2. Sumedha Basu
3. Ayan Nandi
4. Sayan Das

Raspberry Pi 4B

Pi Camera V2

Event: **TRAFFIQ**

Challenge Overview

Build an autonomous car using only real-time camera input with no manual control allowed. The system must:

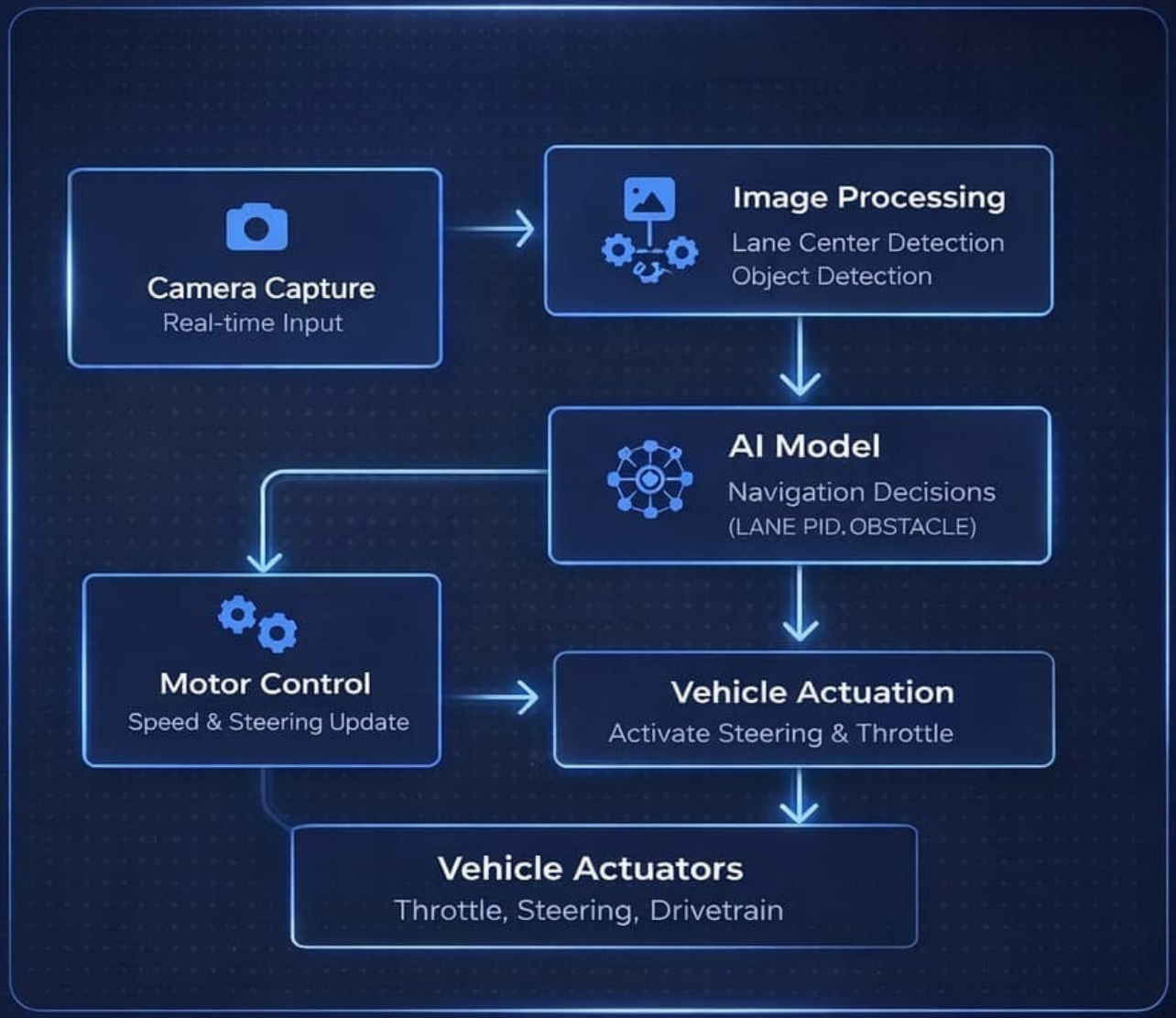
- Detect drivable path - Identify lanes and boundaries in real-time
- Avoid obstacles - Detect and navigate around static/dynamic obstacles
- Make navigation decisions Determine steering and speed in real-time
- Process camera frames - Handle 20-30 FPS for smooth operation

Input
Camera frames (320×240)

AI
Decision making

Output
Motor control

System Flow



PROPOSED SOLUTION

CV + ML Approach

Computer Vision + ML

We use Computer Vision + ML to process camera frames in real-time on Raspberry Pi, enabling fully autonomous navigation:

1. Real-time frame processing - Capture and analyze 20-30 FPS from Pi Camera
2. CNN/Rules-based detection - Identify lanes, obstacles, and navigation cues
3. Model prediction - Predict steering angle and throttle commands
4. Autonomous control loop - Execute motor commands with feedback

Target: 20-30 FPS

Raspberry Pi 4B

Control Flow

End-to-end autonomous control

25 FPS

<40ms

TECH STACK & HARDWARE

Raspberry Pi 4B

Software Stack

Python 3.11 Primary programming language with type hints	v3.11
OpenCV 4.8 Computer vision and image processing	v4.8
NumPy 1.24 Numerical computing and array operations	v1.24
TensorFlow Lite Lightweight ML model inference	v2.13
RPi.GPIO Raspberry Pi GPIO control	v0.7

Hardware Setup

Raspberry Pi 4B

4GB RAM | 1.5GHz Quad-core | 64-bit ARM | WiFi/BT

Pi Camera V2

8MP Sensor | CSI Interface | 1080p@30fps | F2.0

Motor Control

ESC + Servo | PWM Control | 5V UBEC

- Active
- Online

Image Processing Pipeline

Capture
Frame capture

→

Resize/ROI
320×240, ROI

→

Colorspace
Grayscale/HSV

→

Edge Detection
Canny/Threshold

→

Features
Lanes/Contours

Input Processing

- Frame capture: 320×240 @ 30fps
- ROI selection: Focus on road area
- Preprocessing: Resize, denoise

Feature Extraction

- Edge detection: Canny algorithm
- Lane detection: Hough transform
- Contour analysis for obstacles

ML Model Strategy

CNN Regression

Predicts steering angle and throttle from visual features. Input: processed frames, Output: [steer, speed]

Classifier (Optional)

Classifies: left/straight/right for decision support

Rule-based Fallback

Activated when model confidence < 0.5

DECISION-MAKING LOGIC & CODE

AI Decision Flow

How the AI makes navigation decisions:

1 Lane Center Detection

Compute lane center vs vehicle center → error calculation

Lane Detection → Center Calc → Error

2 Deviation Analysis

If deviation > threshold → turn toward center

Error > 10% → Turn Left → PID Control

3 Obstacle Detection

Detect obstacles → slow down/avoid

Object Detected → Distance Check → Avoid

Core Code Logic

Frame processing → Model prediction → Actuator control

```
# -------------------- LANE PROCESSING ------------------
def make_coordinates(image, line_parameters):
    slope, intercept = line_parameters

    if abs(slope) < 0.01:
        slope = 0.01

    y1 = image.shape[0]
    y2 = int(y1 * 0.6)

    x1 = int((y1 - intercept) / slope)
    x2 = int((y2 - intercept) / slope)

    return np.array([x1, y1, x2, y2])

def average_slope_intercept(image, lines):
    if lines is None:
        return None
    left_fit = []
    right_fit = []
    for line in lines:
        x1, y1, x2, y2 = line.reshape(4)

        parameters = np.polyfit((x1, x2), (y1, y2), 1)
        slope, intercept = parameters

        if slope < 0:
            left_fit.append((slope, intercept))
        else:
            right_fit.append((slope, intercept))

    lines_out = []
    if len(left_fit) > 0:
        left_avg = np.average(left_fit, axis=0)
        lines_out.append(make_coordinates(image, left_avg))
    if len(right_fit) > 0:
        right_avg = np.average(right_fit, axis=0)
        lines_out.append(make_coordinates(image, right_avg))
    return np.array(lines_out) if lines_out else None
```

Real-Time Targets

Latency Target
< 40ms/frame
✓ Achieved

Frame Rate
25 FPS
✓ Target

Input Resolution
320×240
✓ Optimized

Memory Usage
< 512MB
✓ Efficient

Optimization: TFLite int8 quantization, vectorized OpenCV, minimal I/O blocking

Safety Mechanisms

Watchdog Timer

Frame timeout detection: 500ms threshold → automatic stop

Active monitoring

Sanity Limits

Steering: ±45°, Speed: 0-100%

Bounds enforced

Confidence Gating

Model confidence threshold: 0.5 → stop if below

Uncertainty handling

Failsafe Mechanism

Hardware kill switch + software emergency stop

Dual protection

CHALLENGES & SOLUTIONS

3 Key Challenges

Navigation Challenges & Solutions

Fog/Low Light

Resolved

Poor visibility conditions affecting camera input quality and lane detection accuracy

- **Denoise:** Gaussian blur + median filter for noise reduction
- **CLAHE:** Contrast Limited Adaptive Histogram Equalization
- **Adaptive Thresholding:** Dynamic threshold adjustment

Narrow Corridors

Resolved

Limited space requiring precise navigation and tight control

- **ROI Tuning:** Focused region of interest for better detection
- **PID Gains:** Tighter control parameters for stability
- **Speed Cap:** Reduced velocity for safety

Obstacles/Clutter

Resolved

Static and dynamic obstacles requiring detection and avoidance

- **Contour Detection:** Shape and size analysis for objects
- **Stop/Avoid:** Hysteresis logic to prevent oscillation
- **Motion Cues:** Velocity tracking for dynamic obstacles